

SOFTWARE REQUIREMENTS SPECIFICATION

Next-Generation AI-Driven iOS Music Platform

Production-Oriented Product, Architecture, and Engineering Requirements

Document Version	1.0
Platform	iOS / iPadOS
Primary Language	Swift
UI Framework	SwiftUI
Architecture	Native iOS + Cloud Backend + On-Device AI
Status	Proposed / Product Definition

This document translates the product vision into implementation-oriented functional, non-functional, architectural, security, AI, audio, social, monetization, testing, and acceptance requirements.

Table of Contents

1. Introduction

1.1 Purpose

1.2 Product Scope

2. Product Vision

2.1 Vision Statement

2.2 Product Principles

3. Product Objectives

O1 — Superior Audio

O2 — Algorithmic Control

O3 — Artist Sustainability

O4 — Privacy

O5 — Frictionless Music Management

O6 — Native iOS Experience

4. Scope

4.1 In Scope

4.2 Out of Scope for Initial Release

5. Stakeholders

5.1 Stakeholder Matrix

6. User Roles

6.1 Listener

6.2 Artist

6.3 Label / Distributor

6.4 Administrator

7. System Architecture

7.1 High-Level Architecture

7.2 Architectural Principles

8. iOS Technology Stack

8.1 Swift

8.2 SwiftUI

8.3 AVFoundation

8.4 Core ML

8.5 SwiftData

8.6 SharePlay / GroupActivities

8.7 Supporting iOS Technologies

9. Core Functional Requirements

FR-001 User Registration and Authentication

FR-002 Music Search

FR-003 Music Playback

FR-004 High-Fidelity Audio

FR-005 Recommendation Variance Slider

FR-006 Recommendation Engine

FR-007 Algorithmic Transparency

FR-008 Shuffle

FR-009 Playlist Management

FR-010 Search Within Playlist

FR-011 Smart Offline Downloads

FR-012 Offline Playback

FR-013 Local Music Import

FR-014 Hybrid Library Reconciliation

FR-015 Library Metadata

FR-016 Favorites

FR-017 Listening History

FR-018 Artist Profiles

FR-019 Direct Artist Support

FR-020 User-Centric Revenue Allocation

FR-021 Social Listening

FR-022 Collaborative Queue
FR-023 Now Playing Interface
FR-024 Queue Gestures
FR-025 Haptic Feedback

10. Artificial Intelligence Requirements

10.1 AI Architecture
10.2 AI Privacy
10.3 Recommendation Diversity
10.4 Cold-Start Strategy
10.5 AI Model Lifecycle

11. Data Model

11.1 Core Entities
11.2 Relationships

12. Security Requirements

SEC-001 Encryption
SEC-002 Keychain
SEC-003 API Security
SEC-004 Content Protection
SEC-005 Payment Security

13. Privacy Requirements

13.1 Privacy-by-Design

14. Non-Functional Requirements

NFR-001 Performance
NFR-002 Playback Startup
NFR-003 Database Performance
NFR-004 Battery
NFR-005 Network Resilience
NFR-006 Scalability
NFR-007 Availability
NFR-008 Reliability

15. Accessibility Requirements

15.1 Accessibility

16. UI/UX Requirements

16.1 Primary Navigation
16.2 Dark Mode
16.3 Now Playing
16.4 Interaction Design

17. Error Handling

17.1 Error States

18. Observability

18.1 Monitoring
18.2 Alerting

19. Analytics

19.1 Product Analytics
19.2 Privacy Constraints

20. Recommendation Evaluation

20.1 Evaluation Framework

21. Testing Requirements

21.1 Unit Testing
21.2 Integration Testing
21.3 Audio Testing
21.4 AI Testing
21.5 Security Testing
21.6 Performance Testing
21.7 Accessibility Testing

22. Acceptance Criteria

22.1 MVP Acceptance Criteria

23. MVP Definition

23.1 Phase 1 — Core Music

23.2 Phase 2 — Differentiation

23.3 Phase 3 — Premium Experience

23.4 Phase 4 — Social

23.5 Phase 5 — Artist Economy

24. Key Product Risks

Risk 1 — Music Licensing

Risk 2 — Artist Payout Model

Risk 3 — Catalog Scale

Risk 4 — iOS Exclusivity

Risk 5 — High-Resolution Audio

25. Future Enhancements

25.1 AI DJ

25.2 AI Playlist Builder

25.3 Personal Music Graph

25.4 Music Discovery Map

25.5 Artist AI Tools

26. Success Metrics

26.1 Listener Metrics

26.2 Discovery Metrics

26.3 Artist Metrics

26.4 Technical Metrics

27. Final Product Architecture

27.1 System Summary

28. Conclusion

28.1 Product Positioning

28.2 Strategic Conclusion

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional, non-functional, architectural, security, data, AI, audio, social, monetization, and operational requirements for a next-generation music streaming platform designed exclusively around music consumption and artist discovery.

1.2 Product Scope

The platform will provide high-fidelity music playback, transparent and user-controlled recommendations, intelligent offline management, a unified local and streaming library, advanced playlist management, social listening through native iOS capabilities, direct artist-support mechanisms, privacy-preserving personalization, and a native iOS experience.

2. Product Vision

2.1 Vision Statement

To build the most focused, transparent, privacy-preserving, and technically sophisticated music platform for iOS, giving listeners greater control over what they hear while creating stronger economic and social connections between artists and fans.

2.2 Product Principles

- Music-first: the core product is playback, discovery, library, and artist connection.
- Algorithmic agency: users can influence recommendation behavior rather than accepting a single opaque feed.
- Privacy by design: personalization should minimize transmission of sensitive listening behavior.
- Native excellence: exploit iOS capabilities instead of treating iOS as a generic deployment target.
- Technical honesty: the interface should report actual audio quality and system state.
- Artist sustainability: commercial mechanisms should aim to improve the economic relationship between listeners and creators.

3. Product Objectives

- **O1 — Superior Audio:** Deliver reliable high-quality playback, including lossless and high-resolution audio where supported by source, device, network, licensing agreement, and subscription tier.
- **O2 — Algorithmic Control:** Allow users to control how familiar or exploratory recommendations should be.
- **O3 — Artist Sustainability:** Create mechanisms through which artists can receive greater economic value from listeners, subject to licensing, payment, tax, and App Store requirements.
- **O4 — Privacy:** Keep sensitive personalization data on-device whenever technically and commercially practical.
- **O5 — Frictionless Music Management:** Provide a unified interface for streaming music, downloaded music, imported local music, playlists, albums, artists, favorites, and listening history.
- **O6 — Native iOS Experience:** Use native Apple technologies for smooth animations, low playback latency, background audio, AirPlay, SharePlay, accessibility, haptics, and efficient resource usage.

4. Scope

4.1 In Scope

- User authentication
- Music catalog browsing
- Music search
- Artist profiles
- Album pages
- Track playback
- Queue management
- Playlist management
- Favorites
- Listening history
- AI recommendations
- Recommendation controls
- Offline downloads
- Smart caching
- Local music import
- Hybrid library reconciliation
- Advanced playlist filtering
- Social listening
- Collaborative queues
- Artist tipping/support
- Subscription management
- Privacy controls
- Accessibility
- Analytics and diagnostics
- Content and rights management

4.2 Out of Scope for Initial Release

- Podcasts
- Audiobooks
- Short-form entertainment video
- General social-media feeds
- Video streaming
- Games
- News
- Non-music entertainment

5. Stakeholders

Stakeholder	Responsibility / Interest
Listener	Music consumption and discovery
Artist	Music distribution and fan engagement
Label	Rights and catalog management

Stakeholder	Responsibility / Interest
Distributor	Content delivery and metadata
Product Team	Product direction
Engineering Team	System development
AI/ML Team	Recommendation systems
Audio Engineering Team	Playback and streaming
Finance / Legal	Revenue accounting, licensing, compliance
Apple / App Store	Platform compliance
Support / Operations	Customer support and system administration

6. User Roles

6.1 Listener

- Search and play music
- Create playlists
- Download music
- Like tracks
- Follow artists
- Control recommendations
- Join listening rooms
- Support artists

6.2 Artist

- Manage artist profile
- Distribute music through approved channels
- Manage metadata
- View listener statistics
- Manage fan-support functionality
- View revenue information

6.3 Label / Distributor

- Submit catalog
- Manage rights metadata
- Manage territories
- Manage release dates
- Manage takedowns
- View distribution information

6.4 Administrator

- Manage users
- Manage content

- Manage rights
- Monitor system health
- Handle abuse reports
- Manage catalog metadata
- Investigate payment and revenue issues

7. System Architecture

7.1 High-Level Architecture

```

iOS Application
↓
SwiftUI + Swift Concurrency + AVFoundation + Core ML + SwiftData + Core Haptics + SharePlay
↓
Secure API Layer
↓
Catalog Service | Recommendation Service | User Service | Rights Service | Subscription/Payment Service | Social
Service | Analytics
↓
Databases + Object Storage + CDN + Monitoring

```

7.2 Architectural Principles

- Separate presentation, domain, data, and infrastructure concerns.
- Use asynchronous work for network, indexing, import, download, and large database operations.
- Treat playback state as a first-class subsystem rather than a normal UI state.
- Use cloud services for catalog-scale operations and on-device services for privacy-sensitive personalization where feasible.
- Design for intermittent connectivity and partial failure.

8. iOS Technology Stack

8.1 Swift

- Primary development language
- Swift Concurrency and async/await
- Actors for concurrency-sensitive state
- Strict type safety

8.2 SwiftUI

The primary UI shall use a state-driven, modular architecture: View → ViewModel/Feature State → Domain Use Case → Repository → Local/Remote Data Source.

8.3 AVFoundation

- Streaming playback
- Local playback
- Background audio
- Audio session management
- Buffer management
- Gapless playback where source material permits
- Audio route management

8.4 Core ML

Core ML shall support on-device personalization and ranking where feasible, including taste embeddings, track similarity, classification, ranking, skip prediction, and preference prediction. The design must not assume a generic 'temperature' control exists inside Core ML; exploration should instead be implemented as an application-level ranking/diversification parameter.

8.5 SwiftData

- User library
- Track metadata
- Playlists
- Listening history
- Download state
- Recommendation state
- Local file references
- Cached metadata

8.6 SharePlay / GroupActivities

- Shared listening sessions
- Synchronized playback state
- Collaborative queues
- Participant management
- Session recovery

8.7 Supporting iOS Technologies

- MediaPlayer / system Now Playing integration
- Network framework for connectivity awareness
- BackgroundTasks for permitted background maintenance
- FileProvider / document access for local music import
- StoreKit for subscription-related platform integration
- Keychain for credentials and secrets
- OSLog and MetricKit for diagnostics and performance telemetry

9. Core Functional Requirements

FR-001 User Registration and Authentication

- Support Sign in with Apple and email-based authentication where appropriate.
- Secure session management and token expiration.
- Account recovery and device/session management.
- Credentials and tokens must use secure iOS storage mechanisms.

FR-002 Music Search

- Search songs, albums, artists, genres, and playlists.
- Support partial matching and typo tolerance.
- Prioritize relevance rather than popularity alone.

FR-003 Music Playback

- Play, pause, resume, skip, previous, seek, repeat, and shuffle.
- Support background playback subject to iOS audio-session requirements.
- Maintain robust playback state across route and network changes.

FR-004 High-Fidelity Audio

Where content licensing and device capabilities permit, support lossless and high-resolution formats. The UI shall display the actual delivered playback quality and shall not claim high-resolution playback when the playback path has been downsampled.

FR-005 Recommendation Variance Slider

Provide a 0-100% recommendation variance control:

- 0-20%: strongly familiar
- 21-50%: balanced
- 51-80%: exploratory
- 81-100%: strong discovery / long-tail emphasis

FR-006 Recommendation Engine

Combine user preference, listening history, track and artist similarity, genre, acoustic features, recency, diversity, exploration weight, availability, and repetition penalties.

Candidate Generation → Ranking → Diversity / Repetition Filter → Personalized Recommendations

FR-007 Algorithmic Transparency

Expose understandable recommendation explanations such as 'Recommended because you frequently listen to alternative rock' without exposing proprietary model parameters.

FR-008 Shuffle

- Standard Shuffle: randomized sequence with repetition avoidance.
- True Shuffle: statistically randomized sequence with explicit repetition constraints.
- Smart Shuffle: balances randomness, preference, and diversity.
- Discovery Shuffle: prioritizes unfamiliar tracks.

FR-009 Playlist Management

- Create, rename, delete, duplicate, and reorder playlists.
- Sort by artist, album, track, date added, BPM, key, duration, and custom order.

FR-010 Search Within Playlist

Search must operate within the selected playlist without forcing a global catalog search.

FR-011 Smart Offline Downloads

Automatic cache management shall consider playback frequency, recency, favorites, playlist membership, storage availability, network conditions, and user-defined limits. Low-value cached content becomes eligible for eviction when storage constraints occur. Users retain manual download controls.

FR-012 Offline Playback

Downloaded content shall remain playable according to licensing and offline authorization rules and shall gracefully handle expired authorization, missing files, corrupted downloads, and incomplete downloads.

FR-013 Local Music Import

Allow supported local audio files to be imported through iOS file-access mechanisms. Candidate formats include MP3, AAC, ALAC, FLAC, WAV, and AIFF, subject to the chosen decoder and playback implementation.

FR-014 Hybrid Library Reconciliation

Use multiple signals: ISRC, audio fingerprinting, metadata similarity, and SHA-256 for exact-file identity. SHA-256 alone cannot identify differently encoded copies of the same recording. User-defined metadata shall be preserved when records are merged.

FR-015 Library Metadata

Track records may include Track ID, ISRC, title, artist, album, album artist, genre, BPM, key, duration, sample rate, bit depth, format, artwork, local file reference, streaming reference, user rating, play count, last played, and date added.

FR-016 Favorites

- Like tracks and albums
- Follow artists
- Save playlists
- Synchronize favorite state across authenticated devices

FR-017 Listening History

Store play count, last played time, skip count, completion percentage, and session history. Provide history viewing, clearing, and a control to disable history-based recommendations.

FR-018 Artist Profiles

- Artist name and biography
- Discography
- Popular tracks
- Albums
- Related artists
- Latest releases
- Fan-support options

FR-019 Direct Artist Support

Provide a business capability for one-time support, artist memberships, merchandise links, or similar mechanisms where legally and commercially supported. Payment implementation must be validated against App Store rules, regional law, tax requirements, and artist agreements rather than assuming Apple Pay is universally applicable.

FR-020 User-Centric Revenue Allocation

Support configurable revenue accounting that can attribute eligible subscription revenue according to individual listener consumption, rights ownership, contracts, territories, distributors, taxes, refunds, chargebacks, and currency conversion. The exact payout formula is a commercial/legal rule, not merely

an engineering constant.

FR-021 Social Listening

- Create listening sessions
- Invite participants
- Synchronize playback state
- Share a queue
- Manage host controls
- Recover from temporary disconnection

FR-022 Collaborative Queue

Participants may add, remove, reorder, or vote on tracks. Permissions shall support Host Only, Host + Moderators, or Everyone.

FR-023 Now Playing Interface

- Artwork
- Track and artist
- Progress and seek
- Playback controls
- Queue
- Like
- Download
- Actual audio-quality indicator
- Artist support
- Licensed lyrics where available
- AirPlay
- SharePlay

FR-024 Queue Gestures

Recommended interactions: swipe right to play next, swipe left to remove, long press to reorder. Gestures must never be the sole means of completing an action; accessible controls shall provide equivalent functionality.

FR-025 Haptic Feedback

Use Core Haptics selectively for actions such as liking a track, queue reordering, download completion, and error/confirmation states. Respect system accessibility preferences such as reduced sensory feedback.

10. Artificial Intelligence Requirements

10.1 AI Architecture

Music Catalog → Candidate Generation → Secure API → Candidate Tracks → On-Device User Profile + Core ML Ranking + Diversity Layer → Personalized Feed

10.2 AI Privacy

- Keep listening history, personal taste profile, recommendation weights, and local music preferences on-device whenever feasible.

- If data leaves the device, minimize it, encrypt it, obtain required consent, define retention, and provide appropriate controls.

10.3 Recommendation Diversity

Apply artist repetition, track repetition, genre diversity, artist diversity, and popularity diversity constraints. Diversity must be optimized rather than maximized blindly; a mathematically diverse feed can still be musically poor.

10.4 Cold-Start Strategy

New users shall receive useful recommendations using onboarding preferences, selected genres/artists, contextual signals that do not require extensive history, and popular/high-quality catalog candidates. The system shall gradually transition toward behavioral personalization.

10.5 AI Model Lifecycle

- Version models and ranking logic.
- Measure model performance before deployment.
- Support rollback.
- Monitor recommendation drift.
- Evaluate fairness across artists, genres, popularity levels, and catalog segments.

11. Data Model

11.1 Core Entities

User, Artist, Album, Track, Genre, Playlist, PlaylistTrack, LibraryItem, LocalFile, PlaybackSession, ListeningEvent, Recommendation, Download, Subscription, Payment, ArtistSupport, ListeningRoom, RoomParticipant, QueueItem, RightsHolder, License.

11.2 Relationships

User → Library / Playlists / Listening History / Recommendations / Downloads / Listening Rooms
Artist → Albums / Tracks / Support Profile
Album → Tracks
Playlist → PlaylistTracks
Track → Artist / Album / Audio Asset / Rights / Metadata

12. Security Requirements

SEC-001 Encryption

Use modern TLS for network communication. Sensitive locally stored information shall use appropriate iOS security mechanisms.

SEC-002 Keychain

Authentication credentials, tokens, and secrets shall not be stored in plaintext.

SEC-003 API Security

- Authentication
- Authorization
- Rate limiting
- Input validation

- Token expiration
- Abuse detection

SEC-004 Content Protection

Streaming and downloaded audio shall implement appropriate content protection and licensing controls. Unauthorized clients must not receive unrestricted direct audio asset URLs.

SEC-005 Payment Security

The application shall not store sensitive payment-card information. Payment processing shall use approved payment infrastructure and comply with applicable platform and regulatory requirements.

13. Privacy Requirements

13.1 Privacy-by-Design

- Control listening history
- Control recommendation personalization
- Control analytics participation
- Control social visibility
- Control profile visibility
- Control downloaded content
- Support account deletion
- Clearly explain collection, purpose, retention, sharing, and deletion mechanisms

14. Non-Functional Requirements

NFR-001 Performance

Target cold-start application launch: ≤ 2.5 seconds under defined benchmark hardware and test conditions.

NFR-002 Playback Startup

Scenario	Target
Local playback	≤ 200 ms
Cached streaming	≤ 300 ms
Network streaming	≤ 800 ms

These values must be measured from user playback command to audible output under controlled and documented test conditions.

NFR-003 Database Performance

For a local library containing 20,000+ tracks, common queries should meet a 100 ms target while large imports and indexing execute asynchronously.

NFR-004 Battery

Measure CPU utilization, energy impact, network activity, device temperature, and battery drain per hour during representative background playback scenarios. Avoid treating a universal '<5% CPU' figure as a sufficient battery requirement.

NFR-005 Network Resilience

- Support variable cellular and Wi-Fi conditions.
- Handle temporary connection loss.
- Handle high latency and packet loss.
- Handle network switching.
- Use buffering and retry strategies that minimize playback interruption.

NFR-006 Scalability

Design backend services for horizontal scaling from approximately 10K users to 100K, 1M, and beyond without fundamental architectural redesign.

NFR-007 Availability

Target $\geq 99.9\%$ monthly availability for critical backend APIs, excluding planned maintenance.

NFR-008 Reliability

Critical operations such as playback state updates, downloads, playlist mutations, and payment records shall be idempotent where appropriate and recoverable after interruption.

15. Accessibility Requirements

15.1 Accessibility

- VoiceOver
- Dynamic Type
- High Contrast
- Reduce Motion
- Reduce Transparency
- Voice Control
- Accessible labels and traits
- Accessible custom actions

The Recommendation Slider shall expose an accessible textual value such as: 'Discovery level: 70 percent.'

16. UI/UX Requirements

16.1 Primary Navigation

Home | Search | Library | Playlists
Persistent Mini Player

16.2 Dark Mode

Dark Mode shall be fully supported. Dark Mode should not be described as a guaranteed battery-saving feature; OLED savings depend on actual luminance, brightness, content, and device behavior.

16.3 Now Playing

The Now Playing screen shall prioritize artwork, track identity, playback controls, queue access, quality information, and artist relationship actions rather than unrelated entertainment content.

16.4 Interaction Design

The UI shall follow native iOS patterns, provide immediate feedback, avoid hidden critical actions, and offer accessible alternatives to gestures.

17. Error Handling

17.1 Error States

Scenario	User-facing response
Network failure	You're offline. Playing downloaded music.
Track unavailable	This track is currently unavailable in your region.
Download failure	Download failed. Check your connection and try again.
Payment failure	Your payment could not be completed.

18. Observability

18.1 Monitoring

- API latency
- Playback failures
- Buffering rate
- Crash rate
- Download failures
- Authentication failures
- Recommendation errors
- Payment failures
- Database latency
- Server resource usage

18.2 Alerting

Critical incidents shall trigger automated alerts with severity, ownership, timestamps, affected components, and diagnostic context.

19. Analytics

19.1 Product Analytics

- DAU / MAU
- Session duration
- Track completion rate
- Skip rate
- Search success rate
- Recommendation acceptance
- Playlist creation
- Download usage

- Buffering rate
- Crash-free sessions
- Subscription conversion
- Artist-support conversion

19.2 Privacy Constraints

Analytics collection shall be limited to legitimate product, reliability, and business needs and shall not become a justification for unnecessary collection of sensitive listening behavior.

20. Recommendation Evaluation

20.1 Evaluation Framework

Dimension	Question
Accuracy	Did the user engage with the recommendation?
Diversity	Are recommendations sufficiently varied?
Novelty	Does the system introduce unfamiliar content?
Coverage	What portion of eligible catalog can be surfaced?
Repetition	How frequently are the same tracks/artists recommended?
Long-tail exposure	How much recommendation exposure reaches less-popular artists?

Click-through or engagement rate must not be the only optimization target because it can reinforce recommendation echo chambers.

21. Testing Requirements

21.1 Unit Testing

- Recommendation calculations
- Playlist operations
- Cache logic
- Metadata reconciliation
- Authentication logic
- Revenue calculations

21.2 Integration Testing

Validate end-to-end flows across iOS application, APIs, backend services, databases, content services, and payment systems.

21.3 Audio Testing

- Lossless playback
- High-resolution playback
- Gapless playback
- Background playback
- Bluetooth

- AirPlay
- External DAC scenarios where supported
- Network interruption

21.4 AI Testing

- Recommendation relevance
- Diversity
- Cold-start users
- New artists
- New releases
- Extreme slider settings
- Sparse listening history
- Heavy users

21.5 Security Testing

- Penetration testing
- API security testing
- Authentication testing
- Authorization testing
- Data leakage testing
- Rate-limit testing
- Payment security testing

21.6 Performance Testing

- Cold start
- Playback startup
- Large library queries
- Import/indexing
- Download throughput
- Memory pressure
- Battery consumption

21.7 Accessibility Testing

- VoiceOver workflows
- Dynamic Type at large sizes
- Reduce Motion
- Voice Control
- Keyboard/assistive navigation where applicable

22. Acceptance Criteria

22.1 MVP Acceptance Criteria

Area	Acceptance Condition
Playback	Music starts reliably, background playback works, and queue controls work.
Offline	Authorized downloaded tracks remain playable and failures are handled gracefully.
Discovery	Changing the discovery control measurably changes recommendation behavior.
Anti-repetition	Recommendations and shuffle do not become dominated by a small repeated subset.
Library	Local and streaming tracks coexist and duplicate reconciliation works.
Playlist	Search, sorting, adding, removing, and reordering work.
Social	Listening sessions synchronize playback state and collaborative queue actions.
Privacy	Users can control personalization and request account deletion.
Accessibility	Core workflows can be completed with VoiceOver and Dynamic Type.

23. MVP Definition

23.1 Phase 1 — Core Music

Authentication → Catalog → Search → Playback → Library → Playlist → Favorites

23.2 Phase 2 — Differentiation

Recommendation Engine + Discovery Slider + Smart Shuffle + Local Music + Hybrid Library

23.3 Phase 3 — Premium Experience

Lossless + High-Resolution Audio + Smart Offline + Advanced Queue + Advanced Playlist Tools

23.4 Phase 4 — Social

SharePlay + Listening Rooms + Collaborative Queue

23.5 Phase 5 — Artist Economy

Artist Profiles + Fan Support + Revenue Accounting + Artist Analytics

Strategic warning: building all phases simultaneously would create excessive technical, licensing, and operational risk. The MVP should prove the core thesis before the social and artist-economy layers are expanded.

24. Key Product Risks

Risk 1 — Music Licensing

This is likely the largest business risk. A technically excellent streaming application has no viable product without the necessary catalog rights. Requirements must distinguish between technical capability and legally licensed availability.

Risk 2 — Artist Payout Model

Real-world accounting may involve rights holders, labels, publishers, distributors, territories, contracts, taxes, refunds, and chargebacks. The accounting layer should therefore be configurable rather than hard-coded around a simplistic formula.

Risk 3 — Catalog Scale

Global-scale catalog management creates difficult problems in indexing, metadata quality, audio feature extraction, rights management, search, candidate generation, and content delivery.

Risk 4 — iOS Exclusivity

Native iOS optimization can improve quality but reduces addressable market. The business case must demonstrate that the premium iOS segment is sufficiently valuable.

Risk 5 — High-Resolution Audio

Higher quality increases bandwidth, storage, CDN cost, and potentially battery/network usage. Adaptive quality and explicit data controls should be considered.

25. Future Enhancements

25.1 AI DJ

Conversational discovery such as: 'Play something like Radiohead but less depressing and more electronic.'

25.2 AI Playlist Builder

Natural-language playlist creation such as: 'Create a 90-minute late-night coding playlist.'

25.3 Personal Music Graph

Model relationships between artist, genre, album, track, mood, acoustic profile, and listening behavior.

25.4 Music Discovery Map

Explore Genre → Subgenre → Artist → Track relationships visually.

25.5 Artist AI Tools

- Audience insights
- Fan segmentation
- Release performance analysis
- Similar-artist discovery
- Playlist opportunity analysis

Artist-facing AI should remain separated from the consumer listening interface to preserve the pure-music philosophy.

26. Success Metrics

26.1 Listener Metrics

- 30-day retention
- Weekly listening hours

- Search-to-play conversion
- Recommendation satisfaction
- Skip rate
- Repeat listening
- Playlist creation
- Offline usage

26.2 Discovery Metrics

- New artists discovered per user
- Long-tail artist exposure
- Recommendation diversity
- Recommendation repetition rate

26.3 Artist Metrics

- Artists receiving revenue
- Revenue per active artist
- Fan-support conversion
- Listener-to-artist engagement

26.4 Technical Metrics

- Playback failure rate
- Buffering ratio
- Median playback startup time
- Crash-free sessions
- Battery consumption
- API latency

27. Final Product Architecture

27.1 System Summary

```

LISTENER
↓
SwiftUI iOS App
├─ Playback: AVFoundation
├─ Local AI: Core ML
├─ Local Data: SwiftData
├─ Haptics / Accessibility / SharePlay
↓
Secure API Layer
↓
Catalog | Recommendation | User/Artist | Rights | Payments | Social | Analytics Services
↓
Database | Object Storage | CDN | Monitoring

```

28. Conclusion

28.1 Product Positioning

The platform is positioned around a different philosophy from conventional streaming services: the application should optimize for music quality, discovery quality, listener control, privacy, and artist

relationships rather than maximizing the number of entertainment categories inside one application.

28.2 Strategic Conclusion

The strongest potential moat is not lossless audio alone. Competitors can offer lossless audio. The more defensible combination is high-fidelity playback + controllable discovery + anti-repetition recommendation + privacy-preserving personalization + hybrid local/streaming library + artist-centric economics.

The technically defensible architecture is hybrid rather than purely on-device: iOS should own personalization, local intelligence, playback, library intelligence, and privacy-sensitive computation, while cloud infrastructure handles catalog-scale operations, rights, distribution, synchronization, and commercial accounting.

This SRS is therefore intended to serve as a foundation for product planning, system design, engineering estimation, prototype development, and subsequent technical design specifications.